KU LEUVEN

FACULTY OF ENGINEERING TECHNOLOGY

GROUP T LEUVEN CAMPUS

Battle Command

GAME4

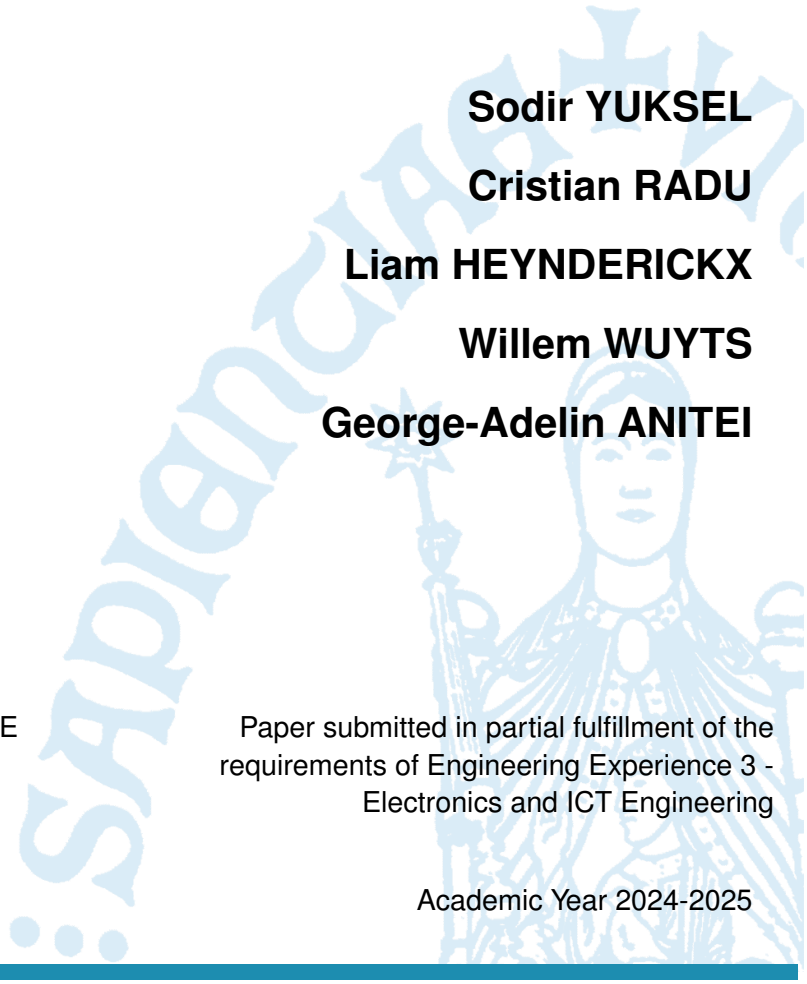
Sodir YUKSEL

Cristian RADU

Liam HEYNDERICKX

Willem WUYTS

George-Adelin ANITEI



Coach: Jit CHATTERJEE

Paper submitted in partial fulfillment of the requirements of Engineering Experience 3 - Electronics and ICT Engineering

Academic Year 2024-2025

KU LEUVEN

Faculty of Engineering Technology Group T Leuven Campus — Paper submitted in partial fulfillment of the requirements of Engineering Experience 3 - Electronics and ICT Engineering
Academic Year 2024-2025

Battle Command

GAME4

ANITEI George-Adelin, WUYTS Willem, HEYNDERICKX Liam, RADU Cristian, YUKSEL Sodir

Bachelor in Engineering Technology: Electronics and ICT Engineering, Faculty of Engineering Technology, Group T Leuven Campus, Andreas Vesaliusstraat 13, 3000 Leuven, Belgium

Coach: Jit CHATTERJEE

Engineering Technology: Electronics and ICT Engineering, Faculty of Engineering Technology, Group T Leuven Campus, Andreas Vesaliusstraat 13, 3000 Leuven, Belgium, jit.chatterjee@kuleuven.be

ABSTRACT

Board games have been a culturally significant form of entertainment for a wide array of people for millennia. There is a constant cycle of new games being developed, and old games being forgotten. In this paper, we will present a re-imagined Battleship board game titled ”Battle Command”. This modern interpretation of a classic board game is designed to be accessible and engaging for a wider audience. By using a visual representation of the game on a physical LED grid, coupled with voice and image recognition, Battle Command enables any user to freely play the game through intuitive hands-free commands. Along with the physical board, there is a paired web-interface to give visual clues, provide game control options and play sound effects to make the experience more immersive. Our aim is to bridge the gap between traditional board games with modern, assisting technologies to create a fun and seamless experience.

©Copyright KU Leuven — This paper is a document that has not been corrected for any errors. Without written permission of the course coordinator and the author(s), it is forbidden to reproduce or adapt in any form or by any means any part of this work. Requests for obtaining the right to reproduce or utilise parts of this work should be addressed to KU Leuven, Group T Leuven Campus, Andreas Vesaliusstraat 13, 3000 Leuven, +32 16 30 10 30 or via e-mail fet.groupt@kuleuven.be A written permission of the course coordinator is also required to use the methods, products, schematics and programs described in this work for industrial or commercial use, and for submitting this work in scientific contests.

1 INTRODUCTION

Board games have been a beloved part of human culture for thousands of years, with one of the earliest known games, "Ur," dating back over 4,000 years (Amlen (2022)). These games transcend mere entertainment, offering intellectual and social stimulation that foster cognitive skills, encourages strategic thinking, and strengthen interpersonal connections (Noda et al. (2019), Fjaellingsdal and Klockner (2020)). However, traditional board games often pose accessibility challenges for certain groups of players or do not resonate with audiences accustomed to modern, technology-enhanced experiences. Addressing these limitations calls for innovative re-imagining of classic games, mixing tradition with technology.

The project "Battle Command," aims to recreate the classic Battleship game to deliver an enriched, accessible experience. Players can engage with Battleship in a novel manner through voice recognition, image detection, and a physical LED grid, enabling broader accessibility by eliminating the need for manual control. This approach combines the tactile and social benefits of a physical board game with the dynamic and adaptive capabilities of modern technology, offering a fresh perspective on a beloved classic.

This paper outlines the design of Battle Command by detailing the hardware and software technologies used and examining the challenges of integrating them into a unified system. It begins with a clear specification of system requirements and then proceeds to a comprehensive discussion of its design and implementation.

By bridging classic gameplay with modern adaptive technologies, we explore how improving traditional games can make them more accessible, engaging, and enjoyable for all.

1.1 Requirements analysis

This section presents the hardware, software, and operational requirements of the Battle Command prototype. The primary objective of the Battle Command system is to provide an authentic gameplay experience while integrating new technologies to create an enhanced user experience.

1.1.1 Hardware Architecture

The system implements a master-slave microcontroller node architecture. An ESP32 microcontroller serves as the master node, with a PIC18F microcontroller functioning as the slave node. These microcontrollers communicate over a wireless connection hosted by nRF24 modules. The physical game board must be of convenient size, allowing portability while maintaining playability. For this reason, the dimensions were established at 22x22 centimeters, comparable to the size of a laptop. Additional hardware requirements include sufficient LED visibility and effective voice recognition for users.

1.1.2 User Interaction

Players play the game primarily through voice commands. These commands determine which position the player targets during gameplay, requiring accurate voice recognition to correctly interpret player input. Alternatively, users can play the game directly in the accompanying web application. The game state is visualized on a 10x10 physical board. The positions of the boats on the board are represented by specific colors using RGB LEDs, providing a clear and intuitive playing experience. This physical visualization enhances the tactile quality of the gameplay experience.

1.1.3 Software and Game Logic

A custom web application accompanies the physical board which is used to play the game. This application allows users to select game difficulty and start the game. Additionally, the web application hosts the core game logic. At game initialization, the position of player placed boats is determined using computer vision. A camera module processes the initial board state, after which an object detection algorithm determines the positions chosen by the player.

1.1.4 System Performance

To ensure smooth gameplay, the system must handle invalid commands gracefully without crashing or malfunctioning. All interactions need to occur in real-time to deliver a fluid and engaging user experience.

1.1.5 Budget Constraints

Beyond technical requirements, the project also financial constraints. The Battle Command prototype must be produced within a budget of 65 Euros.

2 DESIGN AND MATERIALS

The primary design goal of the Battle Command system is to allow users to play a complete game of battleship hands-free against an autonomous opponent. The following section outlines the general design and materials needed to realize this design goal. Each functional component of our system will be discussed separately.

2.1 General Architecture

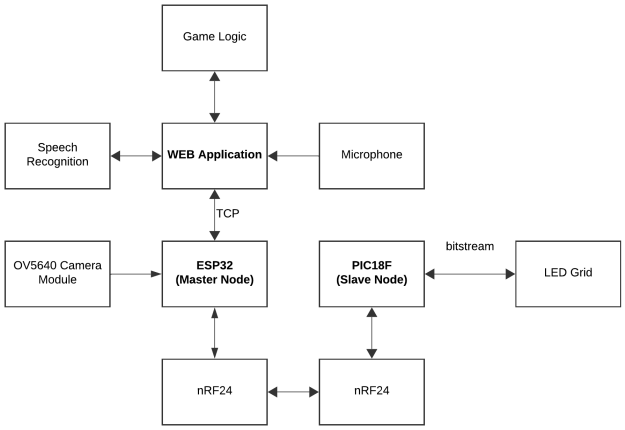


Figure 2.1: Full block diagram for battle command

The Battle Command system follows the architecture outlined in the block diagram in Figure 2.1. The system is divided into three main functional blocks: the Web Application, the ESP32 master node, and the PIC18F slave node. The Web Application is responsible for controlling all game logic, microphone input, speech recognition and is responsible for the image processing. The ESP32 handles communication between the website and hardware via a TCP connection, controlling the camera module, and sending data to the PIC18F. The PIC18F then manages the LED Grid that visualizes the game state to the user.

2.2 Playing Board

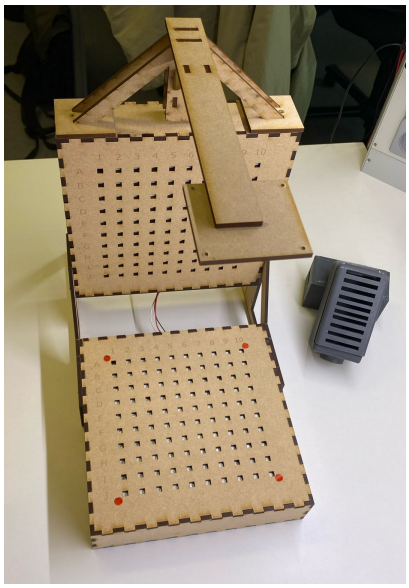


Figure 2.2: MDF Playing Board

The physical playing board was constructed using two 22x22x5 cm boxes. These boxes house the electronics internally, and have the LED grids used for game visualization on the outside. The boxes are constructed using laser-cut MDF, selected for its strength, durability, and cost-effectiveness, making it ideal for prototype development. The board will then be constructed as shown in Figure 2.2, with the user board making up the base and the opponent board forming the vertical part.

2.3 Microcontrollers

The microcontroller setup follows a Master-Slave architecture, with an ESP32 serving as the master node and a PIC18F as the slave node. This allows the different functions of the system to be controlled separately by each microcontroller. All microcontrollers are programmed in C.

The ESP32 is responsible for controlling the camera module during game initialization. After initialization, it handles all communication between the physical playing board and the website. The PIC18F is responsible for controlling the two LED grids that visualize the game state. Communication between the master and slave nodes utilizes a 2.4GHz wireless connection facilitated by nRF24L01 transceivers. These transceivers connect via pins to the PIC18F and ESP32.

2.4 Camera Module

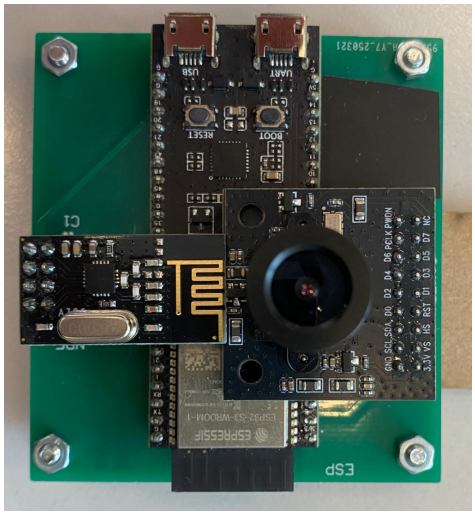


Figure 2.3: Master Node Module with ESP32, OV5640 and nRF24

To obtain the initial state of the Battle Command game, the system requires an image of the board to capture the positions of players' boats. This image is captured by an OV5640 camera module connected to the ESP32 micro-

controller. As shown in Figure 2.3,

2.5 Image Processing

After the image has been captured, it needs to be processed to determine the positions player placed boats. This will be done using a Python program that utilizes the openCV library, which will apply both grid and color detection to determine the position of each boat. This processing is done within the web application.

2.6 LED grid

The LED grid visualizes the Battle Command game state. The grid is constructed using WS2812B LED strips divided into ten segments per board, each containing ten LEDs. Pre-made LED strips were chosen over constructing a custom ten by ten LED grid due to PCB size requirements. Creating a grid matching the playing board would require an expensive PCB, exceeding the budget constraints. Figure 2.4 shows the implemented LED grid with our case

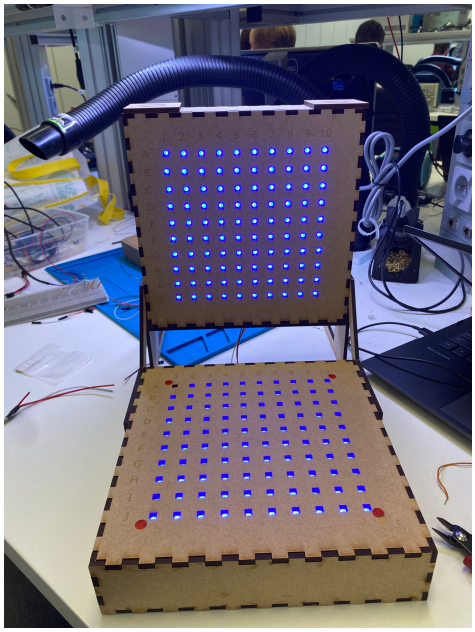


Figure 2.4: LED grid When On

2.7 Microphone Input

Users interact with the game through voice commands. The system uses an electret microphone combined with a preamplifier and a low-pass filter. The preamplifier boosts the raw audio signal, while the low-pass filter attenuates high-frequency background noise, ensuring a cleaner and intelligible speech signal. To minimize noise and ensure the assembly is physically robust, a dedicated PCB was used to implement the circuit. After amplification and processing, the audio signal is sent to the web application Figure 2.5 illustrates the voice input processing workflow.

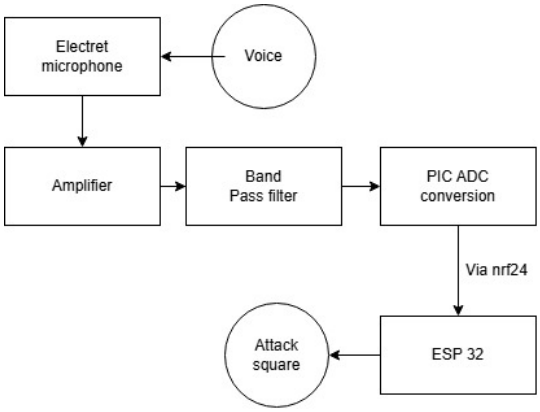


Figure 2.5: Block Diagram voice processing

2.8 Web Application

The web application hosts the game logic and control systems. It enables users to start a game and select difficulty levels, which affects the computer opponent's accuracy. The website also prompts users to capture an image of their playing board, sending a command to the ESP32 to activate the OV5640 camera module. During gameplay, the website manages all game logic, determining if targeted squares contain boats and tracking hits and misses. It also hosts the opponent AI, which uses a probability algorithms to predict optimal squares to target. Beyond these aspects, the website provides additional game visualization, displaying the playing board with animations for hits and misses. A sample interface from an in-progress game is shown in Figure 2.6.

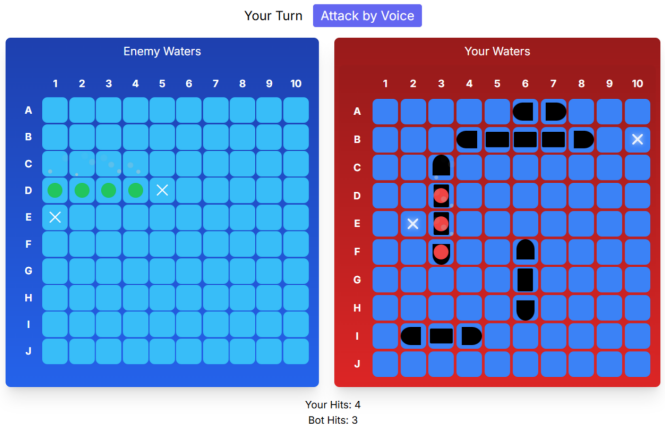


Figure 2.6: Website Game Visualization

3 IMPLEMENTATION

The following section describes how the different modules discussed in the design and materials section are implemented, detailing how the system functions.

3.1 Playing Board

The physical playing board is constructed from laser-cut MDF, assembled with screws and glue. Both boards feature ribbed sides, allowing them to join together easily. A stand, also constructed from laser-cut MDF, connects the two boards as shown in Figure 2.2. The camera module is mounted in a crane-like structure attached to the upper board. This design intentionally resembles a traditional Battleship board, so that the familiarity of the classic game can be preserved.

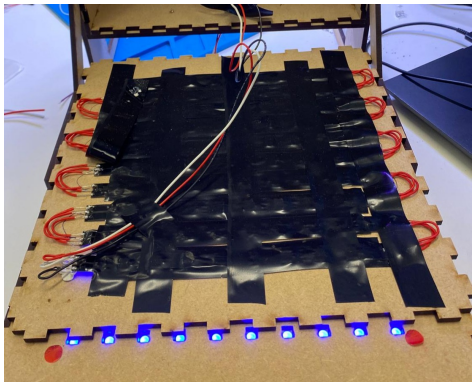


Figure 3.1: LED grid soldered and attached to the board

The primary function of the playing board is to provide visualization of the game state. This visualization is implemented through two WS2812B LED grids (Figure 3.1) representing the game squares. Each LED indicates a specific game state: green for hits, red for misses, and blue for idle squares. The LED grid implementation connects directly to the PIC18F microcontroller, which receives state information from the ESP32 via the wireless nRF24 connection. Figure 3.2 shows the circuit schematic for the LED strips, while Figure 2.4 demonstrates the board with all LEDs in the idle state.

Each WS2812b LED element uses an 8-bit-per-channel (24-bit total) RGB driver and latch circuitry, arranged in a unidirectional serial chain. A single microcontroller I/O pin provides the high-speed data stream of the game we want the LEDs to represent.

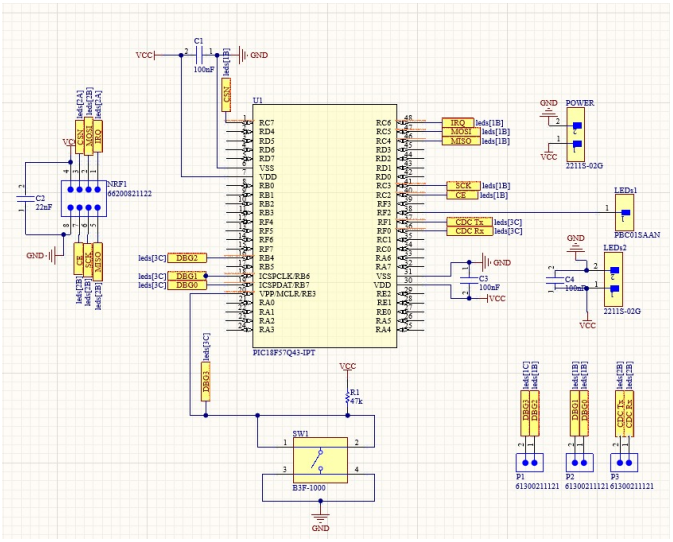


Figure 3.2: LED and PIC18F Circuit Schematic

3.2 ESP32 to PIC18F Communication

The nRF24 modules are responsible for all communication between the ESP32 master node and the PIC18F slave node. These modules connect directly to each microcontroller via GPIO pins, as shown in Figure 3.2 and Figure 3.3. The communication protocol operates through an interrupt-driven system: when the nRF24 receives data, its IRQ pin transitions to a low state, triggering an interrupt that initializes a data read operation on the connected microcontroller.

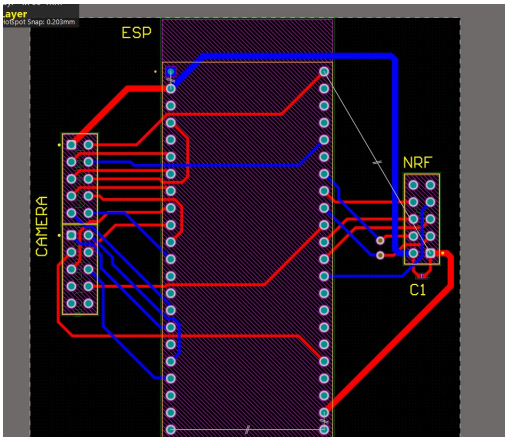


Figure 3.3: PCB Camera and nRF connections

3.3 Camera Module

The camera module captures a high resolution image of the game board to establish the initial game state. An OV5640 camera is used while directly connected to the ESP32's GPIO pins. When the ESP32 receives a "CAPTURE" command through the TCP socket (port 8080), it captures a frame, transmits the image data over the

socket, and returns the frame buffer before closing the client connection. Once transmitted, it undergoes processing to determine boat positions.

To mitigate noise and connection issues, the camera interfaces with the ESP32 through a custom PCB shown in Figure 3.3. The camera initialization configures the device in JPEG capture mode with VGA resolution. Due to the nature of the camera, other settings such as PSRAM frame buffering with auto exposure and white balance adjustments were setup. These settings created the best possible settings to take clear pictures. Figure 3.4 shows a sample image captured by the OV5640 module.

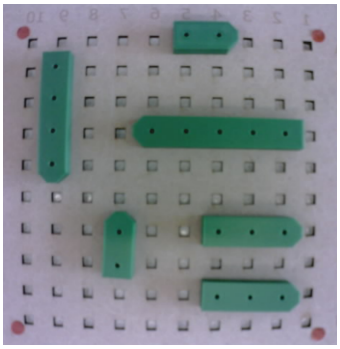


Figure 3.4: Image Captured by OV5640 Camera

3.4 ESP32 PCB

A custom PCB was made to implement the OV5640 camera module and nRF24 transceiver with the ESP32. This PCB assembly is housed in the camera stand (see Figure 2.2), positioning the camera correctly for board image capture during game initialization to ensure that the initial game state will be loaded correctly. Figure 3.3 illustrates the PCB design and the connections between the ESP32, nRF24, and OV5640 camera module.

3.5 Boat Detection

The image processing system receives the camera captured image through the web interface and uses OpenCV for boat detection. The detection algorithm follows a two stage approach:

First, the system performs board segmentation by identifying four red reference dots at the board corners. Using these reference points, the algorithm applies perspective transformation (warping) to normalize the image orientation, ensuring the board corners align with the corners of the 400x400 pixel VGA image. This calibration compensates for potential camera misalignment.

After normalization, the algorithm divides the image into a ten by ten grid and applies color masking techniques to identify boat positions. Each detected boat is mapped to corresponding grid coordinates, which are then transmitted to the game state manager. Figure 3.4 shows the input image, while Figure 3.5 shows the processed output with identified boat positions.

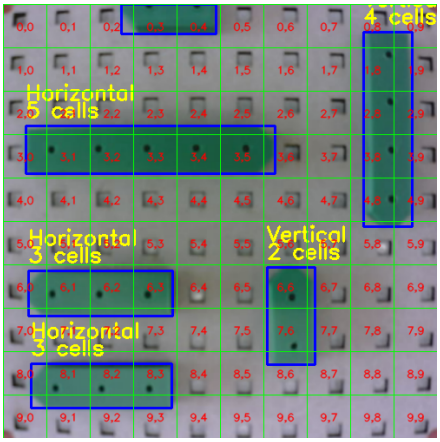


Figure 3.5: Processed Image

3.6 Microphone Circuit

The microphone circuit makes use of an electret microphone that connects to a PCB consisting of a preamplifier and a filter, the schematic of this PCB can be seen in Figure 3.6. The electret microphone generates low-amplitude voltages that are routed into the low-noise amplifier to increase signal strength while minimizing added noise. The amplified analog output is subsequently digitized by connecting it directly to the PC that hosts the Web Application. Once a complete sequence corresponding to a single utterance has been received, the host reconstructs the data into a WAV-format audio file for voice-processing.

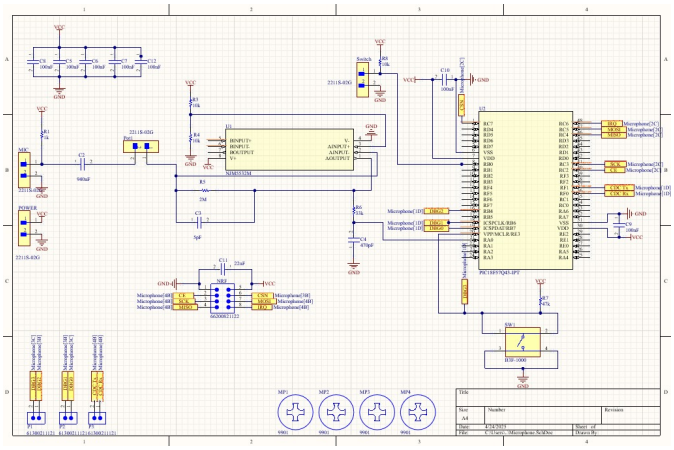


Figure 3.6: Microphone Circuit Schematic

3.7 Speech Recognition

Once the audio has been converted to a wav-format file, it is sent to the Google Speech-To-Text api for processing. This api is native in React (the framework the Battle Command web application is developed in) and thus is accessed directly. Once the audio is processed, the web application receives a string of text containing the grid position spoken by the user.

3.8 Wireless Communication

For the Google Speech-To-Text api to work, the web application requires internet connectivity. For this reason, the WIFI access point of the ESP32 cannot be used. To solve this issue, the connection between the web application and the master node is facilitated through a Wi-Fi connection, in our case, the hotspot of a smartphone. This does not change the mode of connection between the two systems, but it does change the network architecture.

3.9 Web Application

The web application employs a modern full-stack architecture. It integrates the physical gameplay of Battle Command with a digital interface. The application is built with a React frontend, using ‘useState‘ and ‘useEffect‘ hooks for state and lifecycle management. It communicates with backend API endpoints to process board images and control LED hardware. A 10x10 matrix represents both the player’s and AI’s boards, with unique values denoting ship positions and empty cells (see Figure 2.6).

The web application interface provides a structured user flow: image capture or upload, difficulty selection, and live gameplay. It supports user input via grid interaction or the Web Speech API for voice commands. All transitions and visual updates are handled through React state.

The game is initialized with the user capturing an image of their physical board. This is done via an API route that establishes a TCP connection with the ESP32 to retrieve the board image. This image is sent to an API endpoint, where the image-processing algorithm extracts boat positions and returns a structured board matrix. If image parsing fails, the user is prompted to retake the image. The processed board is stored in state and rendered for visual reference.

Two AI difficulty modes are supported, each driven by separate algorithms. Normal mode randomly selects targets and targets adjacent squares upon a hit. Hard mode calculates a probability map based on existing hits and misses, targeting the highest-likelihood cell. State variables maintain attack history, game progression, and bot strategy data, ensuring responsive and consistent UI updates.

Real-time interaction with physical hardware is supported through an API that updates an LED matrix reflecting the current board state, hits, misses, and game end sequences. As with the image capture, this communication is supported though a TCP connection with the ESP32.

4 EVALUATION

4.1 Accuracy

The accuracy of the Battle Command system was evaluated across its key components: the LED visualization grid, voice recognition system, and camera-based object detection.

The LED grid correctly represented the game state in all test cases, with each of the 10×10 positions correctly displaying the appropriate color based on game events (blue for idle state, green for hits, red for misses). The one-to-one mapping between game coordinates and physical LED positions ensured consistent visual feedback throughout gameplay, with no grid squares showing incorrect information.

However, the accuracy of the voice recognition system was condition dependent. Testing revealed several factors affecting recognition performance. The microphone isn’t always accurate due to environmental noise. Accents of users significantly affected the voice recognition as well. Non-native speakers experienced more difficulty with the voice command system than native speakers.

The camera-based boat detection system was similarly affected by environmental factors. The OV5640 camera module proved adequate under optimal lighting conditions but suffered significantly in suboptimal scenarios. Due to the camera’s rather elementary quality, it had issues taking pictures when the lighting and shadow on the board were bad, providing insufficient white balance to ensure image clarity. This would lead to the Python script not being able to identify the boat positions correctly, creating issues with game initialization for users.

4.2 Speed and Response Time

System responsiveness was evaluated to ensure a seamless gameplay experience. Overall, the system provides rapid feedback to maintain player engagement.

Voice command processing occurs quickly enough that users perceive it as instantaneous, however, its perfor-

mance is dependent on the connection speed of the wireless communication discussed in Section 3.8, as well as the speed of the Google Speech-To-Text API. Therefore, taking response time measurements is not indicative of system performance. In cases of poor internet speed, voice processing took noticeably longer, slowing down gameplay. The LED state updates after a game state change are effective almost immediately, showing no perceivable delay. One improvement could be to add animation to the updated LED to make it clearer to users that a color change has occurred. The Web application provides immediate feedback to player actions and has no measurable delay.

A noticeable delay in the gameplay flow comes from the initial camera-based setup process, which requires processing time to retrieve the image and identify boat positions. Despite this initialization delay, the gameplay experience itself remains fluid with near instantaneous feedback while playing. In figure 4.1 the response time of an image capture along with the size of the image transferred can be seen. There is very little correlation between the response time and image size, indicating the delay is caused mainly by external factors, such as the connection speed and hotspot reliability.

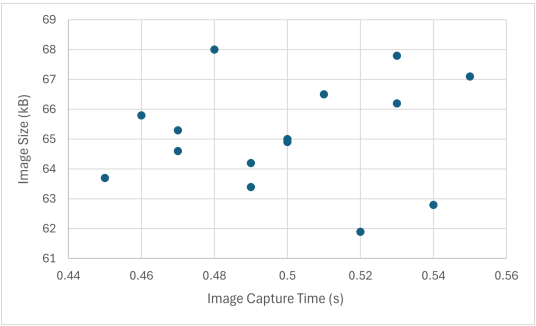


Figure 4.1: Graph showing Image Capture Speed and Image Size.

4.3 System Testing and Validation

Comprehensive testing of Battle Command was conducted through both component testing and integrated system evaluation. Individual hardware components (LEDs, microphone circuit, camera module) were tested separately to verify functionality. Communication between the ESP32 master and PIC18F slave via the nRF24 wireless connection was verified for reliability. Additionally, complete gameplay sessions were conducted with various users to evaluate the entire system.

Testing was primarily conducted during development by playing complete games and analyzing console logs to monitor command interpretation accuracy, game state transitions, communication between system components, and error occurrences and handling. Similarly, all components were tested individually before integration to ensure they work on their own.

As previously discussed, the hardware components such as the LED grid, camera and microphone all functioned as expected, with some minor environmental instabilities in the camera and microphone. The communication between these components, facilitated by the nRF24 modules, proved to be reliable and no failures in communication were detected. This demonstrates successful communication between the website, master node and slave node and ensures the effectiveness of the chosen architecture (See Figure 2.1).

The evaluation of the website was done through observation of user interactions. In general, users found the website both easy to interpret and visually engaging, often choosing to use the web interface to control the game. One improvement could be to emphasize the use of voice control, as users often chose to play the game manually and in some cases did not even realize that voice control was available.

An important metric to consider is the development cost of the prototype. Excluding shipping costs, the prototype was developed for 35 Euros, considering only raw materials and component costs. This cost can also be decreased significantly if Battle Command is produced on a wider scale as the system is modular, making the system affordable to most players.

The system demonstrated high stability during gameplay testing. Minor issues included occasional voice command misinterpretation, particularly in noisy environments, and rare camera detection failures in low light conditions. Future development should focus on improving these aspects to enhance the overall user experience. The system is also affordable and modular, making production on a larger scale a possibility.

5 DISCUSSION

The main objective of Battle Command was to modernize the traditional Battleship game by leveraging modern technology to enhance the gameplay experience. Battle Command successfully integrates voice recognition, computer vision, and interactive LED visualization to deliver an immersive, hands-free gaming experience. These integrated technological features significantly improve accessibility and facilitate intuitive interactions, aligning closely with current trends favoring multimodal interfaces in enhancing user engagement Dritsas et al. (2025)

A strength of our system is the LED visualization grid, which proved to be effective in accurately representing the game state in real-time, providing clear and immediate visual feedback for our users. The web application also proved to be an effective tool in combination with the physical board, and was engaging and intuitive for users, aside from some users not understanding how to use the voice control effectively. Along with this the system is affordable and modular, appealing to large scale production.

However, there were certain limitations with Battle Command in both the voice recognition and computer vision. These limitations were mostly due to environmental factors. The voice recognition module was particularly susceptible to background noise and variations in user accents, leading to occasional inaccuracies in command interpretation. Similarly, the camera module's object detection capabilities were significantly influenced by lighting conditions.

To address these issues, future research could focus on enhancing the robustness of the voice recognition system, possibly through the incorporation of improved microphones or methods to minimize external noise. Increasing the operating voltage of the microphone circuit beyond 3.3V could also provide more headroom and reduce audio signal distortion. Employing speech-to-text models trained on a broader variety of accents could further enhance command recognition accuracy and upgrading camera hardware to handle suboptimal lighting conditions could greatly improve the user experience.

This research extended beyond gaming, with the goal of demonstrating the potential of integrating various modern technologies to create new ways of making interactive games. This project thus serves as a valuable reference for future developments in areas such as assistive technology and educational tools.

6 CONCLUSION

Battle Command successfully demonstrated that traditional board games can be modernized using current technology to create more accessible gaming experience. The project integrated voice recognition, computer vision, and LED visualization into a functional Battleship system that allows hands-free gameplay while maintaining the tactile aspects that make board games appealing.

The system achieved its main goals within the specified constraints. The LED grid provided reliable visual feedback, accurately representing game states throughout testing. The dual-microcontroller architecture using ESP32 and PIC18F processors effectively distributed tasks while maintaining real-time responsiveness. The web application successfully coordinated all system components and provided an intuitive user interface.

Voice recognition and computer vision components showed both promise and limitations. While the voice control system worked well in quiet environments with clear speech, it struggled with background noise and accent variations. Similarly, the camera based boat detection performed adequately under good lighting but had difficulties with shadows and poor illumination. These environmental dependencies represent the main challenges for real-world deployment.

Despite these limitations, Battle Command validated the core concept of technology-enhanced accessible gaming. The research contributes to the field of accessible gaming by demonstrating how multiple technologies can be integrated cost-effectively. The project proves that modern technology can make traditional games more accessible without losing their character. This work opens possibilities for similar accessibility-focused projects in gaming and education.

Battle Command represents a successful step toward making board games accessible to a broader audience, opening possibilities for inclusive and refreshing gaming experiences.

LIST OF SYMBOLS

Acronyms

NRF	Nordic Semiconductor RF Transceiver
ESP32	Espressif's Microcontroller
PIC18	Peripheral Interface Controller 18-bit
GPIO	General Purpose Input Output
ADC	Analog to Digital Converter
TCP	Transmission Control Protocol
MDF	Medium Density Fiberboard
OPENCV	Open Source Computer Vision Library
PSRAM	Pseudo Static Random Access Memory

BIBLIOGRAPHY

Amlen, D. (2022). The royal game of ur: How to play the oldest board game on record. New York Times (Online). Available: https://www.proquest.com/blogs-podcasts-websites/royal-game-ur-how-play-oldest-board-on-record/docview/2737136117/se-2.

Dritsas, E., Trigka, M., Troussas, C., and Mylonas, P. (2025). Multimodal interaction, interfaces, and communication: A survey. *Multimodal Technologies and Interaction*, 9(1):6–6.

Fjaellingsdal, K. S. and Klockner, C. A. (2020). Green across the board: Board games as tools for dialogue and simplified environmental communication. *Simulation & Gaming*, 51(5):632–652.

Noda, S., Shirotsuki, K., and Nakao, M. (2019). The effectiveness of intervention with board games: a systematic review. *BioPsychoSocial Medicine*.